Lab 3: Component Modelling & Architectural Pattern Selection

Blood Bank Management System (Blood bank inventory & emergency donor matcher) · Team 12 · KP Preetika Setty, PES1UG24AM144

1. System Overview

The system helps a blood bank keep accurate blood stock records and respond quickly to emergency requests. Based on the Lab 1 requirements and use cases, it:

- **Manages inventory:** the bank manager adds and updates blood units (type, quantity, collection and expiry date); units within 3 days of expiry are flagged and expired units leave the available count (FR-002, FR-003).
- **Manages donors:** donors register or update their blood type and location, and accept or decline emergency alerts (FR-004, FR-005).
- **Matches emergencies:** a request is checked against inventory; if stock is insufficient, SMS alerts go to compatible donors within 10 km through an SMS gateway (FR-001).
- **Protects data:** one blood unit must never be allocated to two requests, and donor location data must be encrypted and used only for radius matching (NFR-001, NFR-002).

2. Architecture Selection

The selected architecture is a **Layered Architecture** with three layers: Presentation, Business Logic and Data. Each layer uses only the services of the layer below it, through defined interfaces. It suits this project because its features divide naturally into screens, rules (matching, expiry control, alerting) and stored records, and because the Lab 2 backlog already groups its stories by emergency matching, inventory management and donor management. Components inside the Business Logic layer cooperate as peers, for example Matching calls Inventory, Donor Management and Notification.

3. Justification

- **Separation of responsibilities:** the User Interface only collects input and shows status. Donor Management owns donor data, Blood Inventory Management owns stock and expiry rules, Emergency Request Matching coordinates the stock check and donor search, Notification Service handles SMS alerts, and the Blood Bank Database handles storage.
- **Maintainability, testability, modularity and extension:** every component is reached only through an interface, so it can be tested with a stub of its neighbour (for example, Matching with a stub Inventory API) and changed without affecting others. A new feature, such as the escalation to partner blood banks in the Lab 1 alternate flow, can be added in the Business Logic layer without redesigning the User Interface or the database.
- **Security and privacy:** only business components use the Repository API, so the User Interface never accesses stored data directly. The design gives one place to restrict inventory updates to the authorised bank manager and to keep the ledger transactionally consistent (NFR-001). Donor location is exposed only through the Matching Data API for radius matching, supporting NFR-002.
- **Performance and efficiency:** Matching first queries stock through the Inventory API and searches donors through the Matching Data API only when stock is insufficient. Alerts are then sent in one step through the Alert API, which supports the Lab 1 target of notifying donors within 30 seconds (FR-001).

4. Components and Interfaces

Component	Responsibility	Interfaces
User Interface	Takes emergency requests (blood type, quantity, location), stock updates and donor registration; shows request status.	Requires Donor API, Request API, Inventory API.
Donor Management	Maintains donor profiles (blood type, location), eligibility and accept/decline responses.	Provides Donor API, Matching Data API; requires Repository API.
Blood Inventory Management	Records units (type, quantity, collection/expiry date), flags units near expiry, avoids double allocation.	Provides Inventory API; requires Repository API.
Emergency Request Matching	Checks stock for a request; if insufficient, finds compatible donors within 10 km and triggers alerts.	Provides Request API; requires Inventory API, Matching Data API, Alert API.
Notification Service	Sends SMS alerts to matched donors and logs each notification event.	Provides Alert API; requires Repository API, SMS Gateway API.
Blood Bank Database	Stores the inventory ledger, donor profiles, and notification and response logs.	Provides Repository API.
SMS Gateway (external)	Delivers SMS messages; outside the system boundary.	Provides SMS Gateway API.

Communication example: for an emergency request, the User Interface calls the Request API. Matching checks stock through the Inventory API and, if needed, retrieves eligible donors through the Matching Data API. It then calls the Alert API; Notification Service sends the SMS through the SMS Gateway API and logs the event through the Repository API. Status is returned to the interface.

5. Diagram Notation

Provided and required interfaces use standard UML ball-and-socket notation: a ball (lollipop) is a provided interface, a socket is a required interface, and a joined pair is an assembly connector. **Solid arrows** show requests or data flow from the requiring to the providing component; **dashed arrows** show responses or status updates in the opposite direction. The Repository API appears three times because three components use it. Dashed boxes mark the layer boundaries and the external SMS Gateway.